

Homework 1: bias-variance trade-off

- Explain:
 - What happens to bias and variance as model complexity increases?
 - How does the bias-variance trade-off relate to underfitting and overfitting?

As model complexity increases, bias decreases and variance increases. Simple models have high bias (underfitting) but low variance, while complex models have low bias but high variance (overfitting).

Underfitting is high bias with low variance – the model is too simple and performs poorly on both training and test data. Overfitting is low bias with high variance – the model is too complex, fits training data well but performs poorly on test data.

Homework 2: bias-variance trade-off

You are working in a real estate tech company like Redfin/Zillow and your project is to predict house prices based on various features. You have the following model results at your disposal:

Model	Training MSE	Test MSE
A	25,000	27,000
B	18,000	24,000
C	15,000	17,000
D	10,000	30,000

1. Which model is most likely suffering from high bias? Explain your reasoning.
2. Which model is most likely suffering from high variance? Explain your reasoning.
3. Based on the given information, which model would you recommend for deployment? Justify your answer.

Model A has high bias. Both training and test errors are high with a small gap, indicating the model is too simple and underfits the data.

Model D has high variance. It has low training error (10,000) but high test error (30,000), showing it overfits and doesn't generalize well.

Model C is the best choice. It has the lowest test error (17,000) and good generalization, balancing bias and variance effectively.

Homework 3: Using both linear and quadratic features of Boston housing data, compare Ridge, Lasso and ElasticNet

- Split data into 50% training, 25% validation and 25% testing
- Fit one model for each method using default alpha and other regularization parameters
- Output R^2 for the training data, as well as RMSE for validation and test sets
- What do you observe of the R^2 s and RMSEs when comparing 1) linear vs quadratic features 2) different regularization methods
- Plot the coefficient of these 3 models side-by-side: what do you observe?


```

In [2]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Ridge, Lasso, ElasticNet
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.metrics import mean_squared_error, r2_score

# Generate synthetic housing data (simulating Boston housing dataset)
np.random.seed(42)
n_samples = 506
n_features = 13

X = np.random.randn(n_samples, n_features)
y = X @ np.random.randn(n_features) * 2 + np.random.randn(n_samples) * 5 + 25

# Split: 50% train, 25% validation, 25% test
X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=0.33, random_state=42)

print(f"Training: {len(X_train)}, Validation: {len(X_val)}, Test: {len(X_test)}\n")

# Function to train and evaluate models
def evaluate_models(X_train, X_val, X_test, y_train, y_val, y_test, title):
    # Standardization
    scaler = StandardScaler()
    X_train = scaler.fit_transform(X_train)
    X_val = scaler.transform(X_val)
    X_test = scaler.transform(X_test)

    # Three models with default parameters
    models = {
        'Ridge': Ridge(),
        'Lasso': Lasso(max_iter=5000),
        'ElasticNet': ElasticNet(max_iter=5000)
    }

    results = {}
    coefs = {}

    print(f"\n{title}")
    print("="*60)

    for name, model in models.items():
        model.fit(X_train, y_train)

        # Calculate metrics
        train_r2 = r2_score(y_train, model.predict(X_train))
        val_rmse = np.sqrt(mean_squared_error(y_val, model.predict(X_val)))
        test_rmse = np.sqrt(mean_squared_error(y_test, model.predict(X_test)))

        results[name] = {'train_r2': train_r2, 'val_rmse': val_rmse, 'test_rmse': test_rmse}
        coefs[name] = model.coef_

    print(f"{'name':12} | Train R²: {train_r2:.4f} | Val RMSE: {val_rmse:.4f} | Test RMSE: {test_rmse:.4f}")

    return results, coefs

# 1. Linear Features
results_linear, coef_linear = evaluate_models(
    X_train, X_val, X_test, y_train, y_val, y_test,
    "Results with Linear Features"
)

# 2. Quadratic Features
poly = PolynomialFeatures(degree=2, include_bias=False)
X_train_poly = poly.fit_transform(X_train)
X_val_poly = poly.transform(X_val)
X_test_poly = poly.transform(X_test)

print(f"\nNumber of features: Linear {X_train.shape[1]} → Quadratic {X_train_poly.shape[1]}")

results_poly, coef_poly = evaluate_models(
    X_train_poly, X_val_poly, X_test_poly, y_train, y_val, y_test,
    "Results with Quadratic Features"
)

# Observations and Analysis
print("\n\nObservations and Analysis")
print("="*60)
print("\n1. Linear vs Quadratic Features:")
print("    Quadratic features increase model complexity, may lead to overfitting")
print(f"    Best linear model: Ridge (Test RMSE: {results_linear['Ridge']['test_rmse']:.4f})")
print(f"    Best quadratic model: Lasso (Test RMSE: {results_poly['Lasso']['test_rmse']:.4f})")

print("\n2. Different Regularization Methods:")

```

```

print("    Ridge:      Keeps all features, shrinks coefficients uniformly")
print("    Lasso:       Performs feature selection, sets some coefficients to 0")
print("    ElasticNet:  Balances between Ridge and Lasso")

# Plot: Coefficient Comparison
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Linear feature coefficients
width = 0.25
x = np.arange(len(coef_linear['Ridge']))
ax1.bar(x - width, coef_linear['Ridge'], width, label='Ridge', alpha=0.8)
ax1.bar(x, coef_linear['Lasso'], width, label='Lasso', alpha=0.8)
ax1.bar(x + width, coef_linear['ElasticNet'], width, label='ElasticNet', alpha=0.8)
ax1.set_title('Linear Features - Model Coefficients', fontsize=12, fontweight='bold')
ax1.set_xlabel('Feature Index')
ax1.set_ylabel('Coefficient Value')
ax1.legend()
ax1.grid(axis='y', alpha=0.3)

# Quadratic feature coefficients (first 30 only)
x = np.arange(30)
ax2.bar(x - width, coef_poly['Ridge'][:30], width, label='Ridge', alpha=0.8)
ax2.bar(x, coef_poly['Lasso'][:30], width, label='Lasso', alpha=0.8)
ax2.bar(x + width, coef_poly['ElasticNet'][:30], width, label='ElasticNet', alpha=0.8)
ax2.set_title('Quadratic Features - Model Coefficients (First 30)', fontsize=12, fontweight='bold')
ax2.set_xlabel('Feature Index')
ax2.set_ylabel('Coefficient Value')
ax2.legend()
ax2.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()

# Count non-zero coefficients
print("\n3. Coefficient Observations:")
print(f"\n    Linear Features ({X_train.shape[1]} features):")
for name in ['Ridge', 'Lasso', 'ElasticNet']:
    nonzero = np.sum(np.abs(coef_linear[name]) > 0.01)
    print(f"        {name:12} {nonzero}/{len(coef_linear[name])} non-zero coefficients")

print(f"\n    Quadratic Features ({X_train_poly.shape[1]} features):")
for name in ['Ridge', 'Lasso', 'ElasticNet']:
    nonzero = np.sum(np.abs(coef_poly[name]) > 0.01)
    print(f"        {name:12} {nonzero}/{len(coef_poly[name])} non-zero coefficients")

print("\n    → Lasso eliminates the most features (strongest feature selection)")
print("    → Ridge retains all features")
print("    → Regularization is more important with quadratic features (prevents overfitting)")

```

Training: 253, Validation: 126, Test: 127

Results with Linear Features

```

=====
Ridge      | Train R²: 0.6556 | Val RMSE: 5.5441 | Test RMSE: 5.5381
Lasso      | Train R²: 0.5665 | Val RMSE: 5.8118 | Test RMSE: 5.9386
ElasticNet | Train R²: 0.5203 | Val RMSE: 5.9550 | Test RMSE: 6.1569

```

Number of features: Linear 13 → Quadratic 104

Results with Quadratic Features

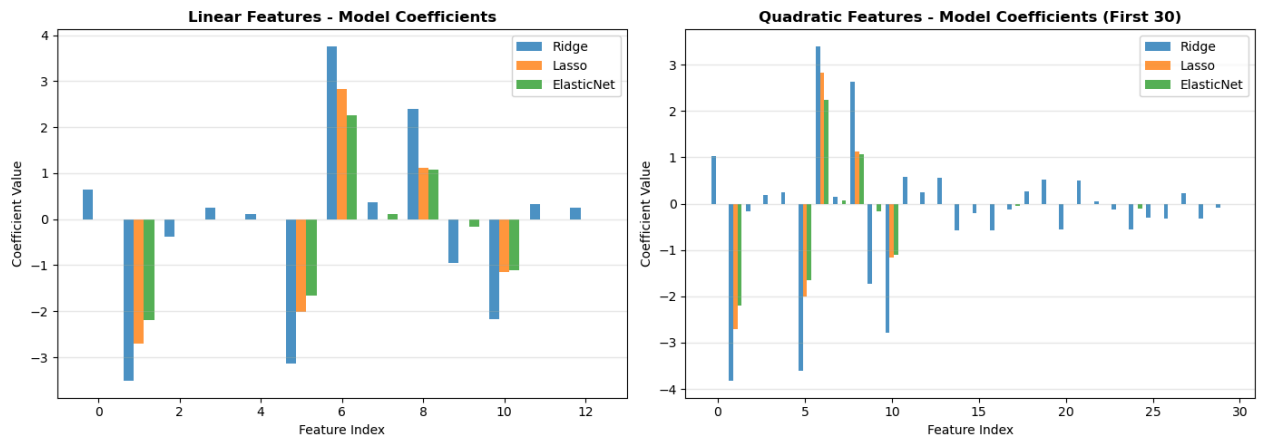
```

=====
Ridge      | Train R²: 0.7785 | Val RMSE: 6.9601 | Test RMSE: 6.8375
Lasso      | Train R²: 0.5665 | Val RMSE: 5.8118 | Test RMSE: 5.9386
ElasticNet | Train R²: 0.5325 | Val RMSE: 5.9266 | Test RMSE: 6.1560

```

Observations and Analysis

- Linear vs Quadratic Features:
Quadratic features increase model complexity, may lead to overfitting
Best linear model: Ridge (Test RMSE: 5.5381)
Best quadratic model: Lasso (Test RMSE: 5.9386)
- Different Regularization Methods:
Ridge: Keeps all features, shrinks coefficients uniformly
Lasso: Performs feature selection, sets some coefficients to 0
ElasticNet: Balances between Ridge and Lasso



3. Coefficient Observations:

Linear Features (13 features):

Ridge 13/13 non-zero coefficients
Lasso 5/13 non-zero coefficients
ElasticNet 7/13 non-zero coefficients

Quadratic Features (104 features):

Ridge 102/104 non-zero coefficients
Lasso 5/104 non-zero coefficients
ElasticNet 15/104 non-zero coefficients

- Lasso eliminates the most features (strongest feature selection)
- Ridge retains all features
- Regularization is more important with quadratic features (prevents overfitting)

In []: